

Software Testing Report: Part 3 White Box Testing and Coverage.

Project: TuxGuitar (Open Source Tablature Editor)

Course: SWE 261P

Member: Xiyao Li & Ping Lu

Date: January 23, 2026 **Base repository link:** [Click](#) **Forked repository link:** [Click](#)

1. Structural (White Box) Testing

1.1 What Structural Testing Is and Why It Matters

Structural (white box) testing designs test cases **from the internal code structure** rather than only from external requirements. It focuses on exercising control-flow elements such as statements, branches, and decision paths. This is important because:

- It reveals **unexecuted logic** that black-box tests may miss.
- It provides **quantitative evidence** of test thoroughness through coverage metrics.
- It helps target **high-risk or complex branches** where defects often hide.

1.2 Coverage Tool and Baseline (Before New Tests)

We ran JaCoCo on the existing test suite in `common/TuxGuitar-lib` and recorded baseline coverage.

Command used:

```
./mvnw -f common/TuxGuitar-lib/pom.xml \
  org.jacoco:jacoco-maven-plugin:0.8.11:prepare-agent \
  test \
  org.jacoco:jacoco-maven-plugin:0.8.11:report
```

Coverage report location: `common/TuxGuitar-lib/target/site/jacoco/index.html`

Baseline coverage (before adding any new tests):

- **Line coverage:** 2,773 / 14,560 = **19.0%**
- **Branch coverage:** 982 / 6,480 = **15.2%**
- **Method coverage:** 713 / 2,786 = **25.6%**

tuxguitar-lib

Sessions

tuxguitar-lib

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
app.tuxguitar.graphics.control		0%		0%	1,636	1,636	3,899	3,899	640	640	26	26
app.tuxguitar.graphics.control.painters		0%		0%	56	56	1,361	1,361	44	44	7	7
app.tuxguitar.song.managers		14%		11%	987	1,093	1,954	2,255	256	294	1	4
app.tuxguitar.player.base		23%		23%	527	656	1,233	1,589	218	281	19	27
app.tuxguitar.util		41%		36%	249	352	423	608	116	156	16	22
app.tuxguitar.song.models		63%		44%	252	683	433	1,378	110	459	2	25
app.tuxguitar.graphics.control.print		0%		0%	168	168	364	364	83	83	5	5
app.tuxguitar.song.helpers		1%		0%	115	119	259	268	46	50	6	7
app.tuxguitar.io.base		8%		1%	158	178	294	338	72	91	6	13
app.tuxguitar.player.impl.sequencer		31%		5%	105	138	201	280	57	89	3	11
app.tuxguitar.util.base64		0%		0%	41	41	132	132	13	13	2	2
app.tuxguitar.ui.resource		0%		0%	96	96	183	183	78	78	8	8
app.tuxguitar.util.plugin		0%		0%	56	56	117	117	41	41	6	6
app.tuxguitar.thread		0%		0%	66	66	130	130	43	43	9	9
app.tuxguitar.graphics.command		0%		0%	49	49	78	78	32	32	5	5
app.tuxguitar.util.properties		0%		0%	54	54	95	95	35	35	4	4
app.tuxguitar.song.helpers.tuning.xml		0%		0%	26	26	83	83	7	7	2	2
app.tuxguitar.action		0%		0%	42	42	85	85	33	33	8	8
app.tuxguitar.song.helpers.tuning		0%		0%	33	33	69	69	26	26	4	4
app.tuxguitar.song.models.effects		59%		33%	28	72	54	158	17	57	0	8
app.tuxguitar.event		0%		0%	29	29	61	61	19	19	5	5
app.tuxguitar.resource		0%		0%	30	30	59	59	19	19	4	4
app.tuxguitar.util.configuration		0%		0%	32	32	51	51	27	27	1	1
app.tuxguitar.io.plugin		0%		0%	20	20	56	56	9	9	3	3
app.tuxguitar.io.tg		97%		95%	16	209	26	746	5	74	1	6
app.tuxguitar.document		0%		0%	10	10	30	30	9	9	3	3
app.tuxguitar.player.plugin		0%		0%	10	10	28	28	6	6	2	2
app.tuxguitar.util.error		0%		0%	12	12	17	17	9	9	2	2
app.tuxguitar.util.singleton		0%		0%	5	5	12	12	3	3	1	1
app.tuxguitar.song.factory		100%		n/a	0	59	0	30	0	59	0	30
Total	62,331 of 74,709	16%	5,498 of 6,480	15%	4,908	6,030	11,787	14,560	2,073	2,786	161	260

Created with JaCoCo 0.8.11.202310140853

1.3 Examples of Currently Uncovered Code

From the baseline report, several packages show **0% line/branch coverage**, indicating large untested areas:

- `app.tuxguitar.graphics.control` (0% line, 0% branch)
- `app.tuxguitar.graphics.control.painters` (0% line, 0% branch)
- `app.tuxguitar.graphics.control.print` (0% line, 0% branch)
- `app.tuxguitar.song.helpers` (1% line, 0% branch)
- `app.tuxguitar.io.base` (8% line, 1% branch)

These packages include rendering/painter logic, printing-related code, and helper utilities that are not currently exercised by the existing unit tests.

1.4 New Test Cases

New test added: `Click common/TuxGuitar-lib/src/test/java/app/tuxguitar/util/base64/TestBase64Codec.java`

Targeted production code:
`app.tuxguitar.util.base64.Base64Encoder`
`app.tuxguitar.util.base64.Base64Decoder`

What the new tests validate:

- **Known Base64 encoding vectors:** We encode `""`, `"f"`, `"fo"`, `"foo"`, and `"foobar"` and compare against the standard Base64 outputs (`""`, `"Zg=="`, `"Zm8="`, `"Zm9v"`, `"Zm9vYmFy"`). This checks correct **padding rules** and the 3-bytes-to-4-chars encoding logic.
- **Known Base64 decoding vectors:** We decode the same Base64 strings and verify we get back the original text. This confirms correct handling of **no padding**, **single padding**, and **double padding** cases.

- **Ignored non-Base64 characters:** We decode `"Zm9v\\nYmFy"` (Base64 with a newline inserted) and verify it becomes `"foobar"`. This confirms the decoder **skips non-Base64 characters** like newlines or whitespace.
- **Binary round-trip on raw bytes:** We encode and then decode a byte array containing edge values like `0`, `127`, and `-1`, and verify the bytes are identical. This ensures **non-text binary data** is preserved without sign/byte loss.

How to run the new test:

```
./mvnw -f common/TuxGuitar-lib/pom.xml -Dtest=TestBase64Codec test
```

After (JaCoCo re-run):

- **Line coverage:** 2,867 / 14,560 = **19.7%** (↑ 94 lines)
- **Branch coverage:** 1,024 / 6,480 = **15.8%** (↑ 42 branches)
- **Method coverage:** 724 / 2,786 = **26.0%** (↑ 11 methods)

tuxguitar-lib

Sessions

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
app.tuxguitar.graphics.control		0%		0%	1,636	1,636	3,899	3,899	640	640	26	26
app.tuxguitar.graphics.control.painters		0%		0%	56	56	1,361	1,361	44	44	7	7
app.tuxguitar.song.managers		14%		11%	987	1,093	1,954	2,255	256	294	1	4
app.tuxguitar.player.base		23%		23%	527	656	1,233	1,589	218	281	19	27
app.tuxguitar.util		41%		36%	249	352	423	608	116	156	16	22
app.tuxguitar.song.models		63%		44%	252	683	433	1,378	110	459	2	25
app.tuxguitar.graphics.control.print		0%		0%	168	168	364	364	83	83	5	5
app.tuxguitar.song.helpers		1%		0%	115	119	259	268	46	50	6	7
app.tuxguitar.io.base		8%		1%	158	178	294	338	72	91	6	13
app.tuxguitar.player.impl.sequencer		31%		5%	105	138	201	280	57	89	3	11
app.tuxguitar.ui.resource		0%		0%	96	96	183	183	78	78	8	8
app.tuxguitar.util.plugin		0%		0%	56	56	117	117	41	41	6	6
app.tuxguitar.thread		0%		0%	66	66	130	130	43	43	9	9
app.tuxguitar.graphics.command		0%		0%	49	49	78	78	32	32	5	5
app.tuxguitar.util.properties		0%		0%	54	54	95	95	35	35	4	4
app.tuxguitar.song.helpers.tuning.xml		0%		0%	26	26	83	83	7	7	2	2
app.tuxguitar.action		0%		0%	42	42	85	85	33	33	8	8
app.tuxguitar.song.helpers.tuning		0%		0%	33	33	69	69	26	26	4	4
app.tuxguitar.song.models.effects		59%		33%	28	72	54	158	17	57	0	8
app.tuxguitar.event		0%		0%	29	29	61	61	19	19	5	5
app.tuxguitar.resource		0%		0%	30	30	59	59	19	19	4	4
app.tuxguitar.util.configuration		0%		0%	32	32	51	51	27	27	1	1
app.tuxguitar.io.plugin		0%		0%	20	20	56	56	9	9	3	3
app.tuxguitar.util.base64		84%		84%	10	41	38	132	2	13	0	2
app.tuxguitar.io.tg		97%		95%	16	209	26	746	5	74	1	6
app.tuxguitar.document		0%		0%	10	10	30	30	9	9	3	3
app.tuxguitar.player.plugin		0%		0%	10	10	28	28	6	6	2	2
app.tuxguitar.util.error		0%		0%	12	12	17	17	9	9	2	2
app.tuxguitar.util.singleton		0%		0%	5	5	12	12	3	3	1	1
app.tuxguitar.song.factory		100%		n/a	0	59	0	30	0	59	0	30
Total	61,641 of 74,709	17%	5,456 of 6,480	15%	4,877	6,030	11,693	14,560	2,062	2,786	159	260

Created with JaCoCo 0.8.11.202310140853